

GINJA

Software Delivery Handbook

From A Working Rig To A Repeatable Workflow

Configure deliberately. Verify the result. Keep operation understandable.

Created By Ry Dos

Public Documentation / Revised October 14, 2026

Application source and executable remain private. Live demonstration available on request.

The Delivery Path

01 Scope And The Operating Workflow 3

02 Environment And A Working Baseline 4

03 Configuration And Saved State 5

04 Validation And Acceptance Evidence 6

05 Troubleshooting And Recovery 7

06 Updates And Change Control 8

07 Handover And Support 9

08 Data, Dependencies And Attribution 10

09 Expansion Through Defined Interfaces 11

The practices describe a proposed delivery approach. Recorded runtime results belong in the validation evidence.

Scope And The Operating Workflow

Define the job before choosing a configuration.

01 / Start With The User Task

Ginja grew from a studio workflow spanning physical controllers, MIDI destinations, production applications and the computer running them. Begin with a task such as selecting a sound and changing an OBS scene from one control. Identify the input, intended actions, receivers and visible outcome. That task becomes the basis for setup, explanation and verification.

02 / Separate Observation From Action

The current application combines monitoring, detachable views, diagnostics and CC-triggered action sequences. A detected device or running process is an observation. A routing rule makes a separate decision about which action to send. Successful transmission still needs confirmation at the receiver: the intended sound, scene, recording or mapped parameter must actually change.

03 / Understand This Handbook’s Scope

Ginja is a working Windows application used in the creator’s workflow. Its source and executable remain private; the public materials explain the design and implementation. The delivery practices here describe a repeatable approach for evaluating a defined environment. They are not a record of customer deployment or completed acceptance across other systems.

> A clear outcome connects the user’s need to a configuration and an observable result.

Environment And A Working Baseline

Begin with one known path; add complexity after it works.

01 / Record The Environment

Identify the application build, Windows version, controller and interface drivers, available MIDI ports and relevant external software. Optional features have their own providers, such as PresentMon for frame observations. Record only requirements supported by the intended workload and actual testing. External applications and drivers remain separately configured components.

02 / Establish One Working Path

Connect one controller and one known destination. Confirm that Ginja receives the expected MIDI source, channel, CC and value. Configure a small action, then check the real result at the receiver. For monitoring, compare a card with the application or device it describes. Save the observed values and steps before adding another device or a longer sequence.

03 / Check Startup And Completeness

In an authorized demonstration or future packaged evaluation, verify that artwork, required modules and any audio-discovery helper load together. A copied executable alone may not represent a complete package. A new environment needs its own startup, port-access and provider checks. Keep the baseline focused enough that another person can repeat the same task.

> The baseline is a reproducible configuration, not simply a dashboard that opens.

Configuration And Saved State

Know which settings change behavior and which change the view.

01 / Give Controls Explicit Jobs

A current routing rule matches an input source, channel, CC and trigger value or Any Value. Each action names its target and parameters. Match observed messages rather than assuming tutorial values. Broad source/value matches should be deliberate. Optional value-change triggering suppresses identical repeats; it is not a general noise filter or automatic feedback-loop detector.

02 / Distinguish Profiles From Routing

Studio and Gamer views select relevant dashboard cards. Custom layout profiles preserve workspace choices such as appearance, watched applications and popout positions. They are separate from routing rules and other service settings. A saved workspace should help an operator find feedback; it should not be described as a complete configuration backup.

03 / Preserve A Recoverable Configuration

Record the working rule fields, destination names and service settings before changing versions or computers. The reviewed application saves OBS host and port while retaining its password for the current run. Recheck persisted settings after restart. A universal settings export/import procedure is not established here, so transfers and restoration need an explicit, tested method.

> Treat input identity, action targets, workspace layout and credentials as separate configuration responsibilities.

Validation And Acceptance Evidence

Agree what success means, then observe the full task.

01 / Define The Expected Result

Choose a representative workflow and write its expected outcome before testing. For a multi-step sequence, identify the result required from every destination and the acceptable response behavior. For example, a sound selection and an OBS scene change must both be checked. Agree how partial completion, unexpected actions and recovery will affect acceptance.

02 / Exercise The Boundaries

Run the normal path, then test the relevant exceptions: repeated values, overlapping rules, a missing port, disconnect/reconnect, unavailable OBS, restart and note release. Pair NOTE ON and NOTE OFF on the intended target, pitch and channel. Test Sequence sends actual actions and bypasses trigger waiting, so test the physical control separately after reviewing the output path.

03 / Keep A Useful Test Record

Record build/environment, steps, expected result, observed result and an evidence reference. Distinguish Pass, Fail and Not Run. Logs show reported steps; receiver observation confirms the task. Documentation rendering and link checks do not establish runtime acceptance. When a feature changes, repeat affected tests and a small working baseline before relying on the new behavior.

> Acceptance rests on the agreed task result in the intended environment, supported by a traceable record.

Troubleshooting And Recovery

Locate the failed stage before changing the system.

01 / Separate The Stages

Start with the symptom and time. Confirm the received event, matching rule, action result and receiver state in order. If sound changes but recording does not start, inspect the recording action and OBS connection rather than changing the controller first. Keep confirmed facts separate from hypotheses, and change one relevant variable before repeating the same task.

02 / Use Evidence With Context

Event history, process readings and session reports help correlate what happened. A high CPU value or a ready indicator does not by itself establish the cause. Preserve a small relevant interval and record the configuration around it. Check driver access, receiver mappings, provider availability and external application state when those layers are involved.

03 / Recover The Operator’s Task

A sequence can stop after earlier actions have completed. Account for destination state and any note that still needs release before repeating it. Use a verified temporary workflow or a compatible known-working build and configuration. Restore operation first, then investigate the underlying cause. Incident management and problem management support these different responsibilities.

> Recovery must consider what already happened, not only the step that failed.

Updates And Change Control

Introduce a bounded change and preserve a way back.

01 / Assess The Change

Describe the user need, affected interfaces and settings, expected behavior and risk. Identify who reviews the decision and what evidence is required. This is the purpose of change enablement: make a deliberate decision about a specific change. Avoid mixing unrelated fixes and new integrations into a single unrepeatable evaluation.

02 / Prepare And Introduce

Release preparation connects an identifiable build with change notes, dependencies, configuration guidance, known limits and recovery instructions. Deployment is the introduction into the selected environment. Verify setup and repeat the agreed task in a limited evaluation before expanding. These are proposed delivery practices; this handbook does not claim an existing automated release pipeline.

03 / Observe And Recover

Compare outcomes with the baseline and collect operator feedback. Preserve the earlier package and check saved-setting compatibility before treating it as a recovery path. An older executable alone may not restore all state. If behavior regresses, contain the affected workflow, communicate the impact and validate the restored task. No automatic updater or cross-device rollback is assumed.

> A useful update includes an observable benefit, a validation record and a recovery decision.

Handover And Support

Make the workflow understandable beyond its original creator.

01 / Teach A Representative Task

Show the purpose of the control, the relevant rule and the result at each receiver. Ask the operator to repeat the task and explain where they would look if it failed. Use the saved layout to keep relevant feedback visible. A short teach-back demonstrates understanding more directly than simply listing every application feature.

02 / Leave The Operating Context

Provide the build identifier, environment, working configuration record, task instructions, known limits and recovery steps. Distinguish a saved layout from service credentials and routing rules. Agree what constitutes successful operation and where evidence is retained. For the current portfolio, the video, presentation, technical reference and visual manual serve different parts of that explanation.

03 / Make A Report Actionable

A useful report contains a minimal reproduction, expected and observed outcomes, versions, impact and a redacted evidence excerpt. Identify the next check and its owner. State support expectations without inventing a response-time commitment. Validate a correction against the original steps before marking it resolved, and update guidance when the result changes the operating workflow.

> A successful handover leaves a person able to operate, explain and investigate the task.

Data, Dependencies And Attribution

Document the boundaries of the interfaces being used.

01 / Describe Data Movement

The reviewed implementation stores preferences, rules, events and session reports locally. Reports can contain device/application names, paths and errors. Optional connections have distinct purposes: OBS uses a configured WebSocket endpoint, ping contacts a selected host, hardware links open external tools, and Discord profile connection and public activity sharing are separate choices.

02 / Know What An Evaluation Includes

Record the dependencies and providers required by the evaluated build. If distribution is later considered, review the actual runtime, bundled libraries and assets, applicable license terms and required notices before selecting a distribution route. This portfolio does not offer an executable or grant an application-use license; its purpose is to explain the working system and delivery method.

03 / Use Evidence Accurately

Separate live observations, saved readings, tutorial fixtures and illustrative screen edits. Identify planned integrations as planned. Product names and third-party visuals describe the interfaces involved; they do not establish affiliation or endorsement. Public reports should contain only the information needed to explain or reproduce the behavior, with credentials and unrelated personal details removed.

Keep operational evidence useful, proportionate and accurate about what it demonstrates.

Expansion Through Defined Interfaces

Extend the method one supported input or destination at a time.

01 / Begin With The Contract

For a proposed feature, define the input event, its source, supported operating mode, intended action and confirmation of success. Decide how authorization, timing, disconnects and recovery should behave. A device picture or architecture arrow communicates an idea; the adapter and test evidence establish the implemented behavior.

02 / Reuse Decisions, Not Assumptions

The current foundation uses source-aware CC rules, ordered MIDI/OBS actions and monitoring feedback. A new product needs its own supported interface and configuration. Push 3 extensions, deeper Ableton integration, Synesthesia, Stream Deck and Xbox-controller workflows remain development directions. The transferable skill is defining the contract and making the workflow testable.

03 / Promote Features With Evidence

Distinguish exploration, implementation, validation and delivered behavior. Add an integration to the current-capability list after its supported scope is implemented and verified. Use operator feedback and observed failure cases to prioritize the next improvement. Keep the evidence tied to a build and environment so later changes can be evaluated against a meaningful baseline.

> An adaptable platform grows through explicit contracts, verified behavior and understandable operation.